

HDOC – Experiment 31

Question

Write a C program to perform the following menu-driven operations on a given 2D array:

1. Find the sum of all elements in a matrix.
2. Find the sum of each row of a matrix and store it in an array.
3. Find the transpose of a matrix.
4. Exit

❖ Algorithm

Step 1: Start.

Step 2: Read the number of rows r and columns c .

Step 3: Validate the dimensions. If $r \leq 0$ or $c \leq 0$, display an error message and ask again.

Step 4: Declare a 2D matrix $a[r][c]$.

Step 5: Read all the matrix elements using nested loops.

Step 6: Repeatedly display the menu:

1. Sum of all elements
2. Sum of each row
3. Transpose of matrix
4. Exit

Step 7: Read the user's choice.

Step 8: Use a switch statement:

- **Choice 1:** Initialize $sum = 0$. Traverse the entire matrix and add every element to sum . Display the total.

- **Choice 2:** Declare a row-sum array. For every row, calculate the sum of its elements and store it in the corresponding position of the row-sum array. Display each row sum and the complete array.
- **Choice 3:** Display the transpose by interchanging the row and column indices while printing, i.e. print `a[j][i]`.
- **Choice 4:** Exit the program.
- **Default:** Display an invalid-choice message.

Step 9: Continue displaying the menu until the user selects 4.

Step 10: Stop.

❖ Test case table

Test Case	Rows × Columns	Matrix	Choice	Expected Result
1	2×3	$\begin{matrix} 1 & 2 & 3 & / & 4 & 5 \\ 6 \end{matrix}$	1	Sum = 21
2	2×3	$\begin{matrix} 1 & 2 & 3 & / & 4 & 5 \\ 6 \end{matrix}$	2	Row sums = [6, 15]
3	2×3	$\begin{matrix} 1 & 2 & 3 & / & 4 & 5 \\ 6 \end{matrix}$	3	Transpose = 1 4 / 2 5 / 3 6
4	1×1	5	1	Sum = 5
5	2×2	$\begin{matrix} -1 & 2 & / & 3 & -4 \end{matrix}$	1	Sum = 0
6	2×2	$\begin{matrix} -1 & 2 & / & 3 & -4 \end{matrix}$	2	Row sums = [1, -1]
7	0×3	—	—	Invalid dimensions; input requested again
8	3×-2	—	—	Invalid dimensions; input requested again
9	Any valid matrix	—	5	Invalid choice!
10	Any valid matrix	—	4	Program terminates

❖ Program code in C

```
#include <stdio.h>

int main()
{
    int r, c, i, j, choice, sum;

    /* Read and validate matrix dimensions */
    do
    {
        printf("Enter the number of rows and columns: ");
        scanf("%d %d", &r, &c);

        if (r <= 0 || c <= 0)
        {
            printf("Invalid dimensions! Rows and columns must be positive.\n");
        }

    } while (r <= 0 || c <= 0);

    /* Variable Length Array */
    int a[r][c];

    printf("\nEnter the matrix elements:\n");

    for (i = 0; i < r; i++)
    {
```

```
    for (j = 0; j < c; j++)
    {
        scanf("%d", &a[i][j]);
    }
}

do
{
    printf("\n1. Sum of all elements\n");
    printf("\n2. Sum of each row\n");
    printf("\n3. Transpose of matrix\n");
    printf("\n4. Exit\n");

    printf("\nEnter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            sum = 0;

            for (i = 0; i < r; i++)
            {
                for (j = 0; j < c; j++)
                {
                    sum = sum + a[i][j];
                }
            }
        }
```

```
printf("Sum of all elements = %d\n", sum);
```

```
break;
```

case 2:

```
{
```

```
int rowSum[r];
```

```
printf("\nRow sums:\n");
```

```
for (i = 0; i < r; i++)
```

```
{
```

```
rowSum[i] = 0;
```

```
for (j = 0; j < c; j++)
```

```
{
```

```
rowSum[i] = rowSum[i] + a[i][j];
```

```
}
```

```
printf("Sum of row %d = %d\n", i + 1, rowSum[i]);
```

```
}
```

```
printf("\nThe row-sum array contains: [");
```

```
for (i = 0; i < r; i++)
```

```
{
```

```
printf("%d", rowSum[i]);
```

```
        if (i < r - 1)
        {
            printf(", ");
        }
    }
```

```
    printf("]\n");
    break;
}
```

case 3:

```
printf("\nTranspose of the matrix:\n");
```

```
for (j = 0; j < c; j++)
{
    for (i = 0; i < r; i++)
    {
        printf("%d\t", a[i][j]);
    }
}
```

```
    printf("\n");
}
break;
```

case 4:

```
printf("Exiting the program...\n");
break;
```

default:

printf("Invalid choice! Please enter a choice from 1 to 4.\n");

}

} while (choice != 4);

return 0;

}

❖ Compile, Execute and Evaluate

```
(kali㉿kali)-[~/Desktop/exp31]
$ vi array4.c

(kali㉿kali)-[~/Desktop/exp31]
$ gcc array4.c -o array4

(kali㉿kali)-[~/Desktop/exp31]
$ ./array4
Enter the number of rows and columns: 2 3

Enter the matrix elements:
1 2 3 4 5 6

1. Sum of all elements
2. Sum of each row
3. Transpose of matrix
4. Exit

Enter your choice: 1
Sum of all elements = 21

1. Sum of all elements
2. Sum of each row
3. Transpose of matrix
4. Exit

Enter your choice: 2

Row sums:
Sum of row 1 = 6
Sum of row 2 = 15

The row-sum array contains: [6, 15]

1. Sum of all elements
2. Sum of each row
3. Transpose of matrix
4. Exit
```

Enter your choice: 3

Transpose of the matrix:

1 4
2 5
3 6

1. Sum of all elements
2. Sum of each row
3. Transpose of matrix
4. Exit

Enter your choice: 4

Exiting the program...

```
(kali㉿kali)-[~/Desktop/exp31]  
$ ./array4
```

Enter the number of rows and columns: 1 1

Enter the matrix elements:

5

1. Sum of all elements
2. Sum of each row
3. Transpose of matrix
4. Exit

Enter your choice: 1

Sum of all elements = 5

1. Sum of all elements
2. Sum of each row
3. Transpose of matrix
4. Exit

Enter your choice: 4

Exiting the program...

```
(kali㉿kali)-[~/Desktop/exp31]
$ ./array4
Enter the number of rows and columns: 2 2

Enter the matrix elements:
-1 2 3 -4

1. Sum of all elements
2. Sum of each row
3. Transpose of matrix
4. Exit

Enter your choice: 1
Sum of all elements = 0

1. Sum of all elements
2. Sum of each row
3. Transpose of matrix
4. Exit

Enter your choice: 2

Row sums:
Sum of row 1 = 1
Sum of row 2 = -1

The row-sum array contains: [1, -1]

1. Sum of all elements
2. Sum of each row
3. Transpose of matrix
4. Exit

Enter your choice: 4
Exiting the program...
```

```
(kali㉿kali)-[~/Desktop/exp31]
```

```
$ ./array4
```

Enter the number of rows and columns: 0 3

Invalid dimensions! Rows and columns must be positive.

Enter the number of rows and columns: 3 -2

Invalid dimensions! Rows and columns must be positive.

Enter the number of rows and columns: 4 4

Enter the matrix elements:

1 2 3 4 5 6 7 8

1 2 3 4 5 6 7 8

1. Sum of all elements
2. Sum of each row
3. Transpose of matrix
4. Exit

Enter your choice: 4

Exiting the program...

```
(kali㉿kali)-[~/Desktop/exp31]
```

```
$
```